

`MPI_Init(&argc, &argv)`

What it does

`MPI_Init` **starts up the MPI execution environment.**

Until you call it, the program is just a normal sequential process. After you call it, your code is running inside the *MPI world* — a coordinated network of processes that can now communicate.

Under the hood, it does several important things:

1. **Spawns or connects processes**

When you run a program with `mpirun -np 4 ./program`, `mpirun` starts four identical copies of your program. `MPI_Init` makes each copy aware that it's part of a larger “parallel job” and sets up connections among them.

2. **Initializes communication channels**

It establishes the messaging fabric — TCP sockets, shared memory buffers, or high-speed interconnects (like InfiniBand), depending on your system.

3. **Parses command-line arguments**

MPI implementations may reserve special flags (e.g., `--mca` in OpenMPI). That's why you pass `&argc` and `&argv` — MPI can consume what it needs and leave the rest for your code.

4. **Sets up internal state**

MPI needs to track process ranks, groups, communicators, and message queues.

`MPI_Init` builds those structures before you start using MPI calls.

After `MPI_Init`, every process is fully connected and ready to communicate.

Until that point, no MPI call is safe.

A **communicator** in MPI is like a social group — it defines *who can talk to whom*.

Every communicator:

- contains a **group of processes**, each with a **rank** unique within that group.
- acts as the *scope* for all communications.

```
int color = world_rank / 4;
MPI_Comm_split(MPI_COMM_WORLD, color, world_rank, &new_comm);
```

So:

- Processes 0–3 share one communicator (`new_comm` for `color=0`).

- Processes 4–7 share another (`new_comm` for `color=1`).
- Each group can talk internally using `new_comm` , but cannot directly communicate across groups.

`key` defines the **ordering** of ranks inside each subgroup. If all processes in a group have the same `color` , then MPI sorts them by `key` to assign new ranks.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int bigdata[1000000];

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 3) {
        if (rank == 0) printf("Run with 3 processes.\n");
        MPI_Finalize();
        return 0;
    }

    int next = (rank + 1) % 3;
    int prev = (rank + 2) % 3;

    printf("Process %d sending to process %d\n", rank, next);
    MPI_Send(bigdata, 1000000, MPI_INT, next, 0, MPI_COMM_WORLD);

    printf("Process %d receiving from process %d\n", rank, prev);
    MPI_Recv(bigdata, 1000000, MPI_INT, prev, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    printf("Process %d finished\n", rank);

    MPI_Finalize();
    return 0;
}
```

How to Handle ?

****Method 1: Reverse the order for one side**

```

if (rank == 0) {
    MPI_Recv(bigdata, 1000000, MPI_INT, prev, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    MPI_Send(bigdata, 1000000, MPI_INT, next, 0, MPI_COMM_WORLD);
} else {
    MPI_Send(bigdata, 1000000, MPI_INT, next, 0, MPI_COMM_WORLD);
    MPI_Recv(bigdata, 1000000, MPI_INT, prev, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
}

```

****Method 2: Use non-blocking communication (MPI_Isend , MPI_Irecv)**

```

MPI_Request request;
MPI_Status status;

MPI_Isend(bigdata, 1000000, MPI_INT, next, 0, MPI_COMM_WORLD, &request);
MPI_Irecv(bigdata, 1000000, MPI_INT, prev, 0, MPI_COMM_WORLD, &status);
MPI_Wait(&request, &status);

```

Here, `MPI_Isend` *starts* sending but returns immediately — it doesn't wait for the receiver. Then the process can post its `MPI_Irecv` . Finally, `MPI_Wait` ensures the send actually completes before proceeding.

****Method 3: Use MPI_Sendrecv**

```

MPI_Sendrecv(
    bigdata, 1000000, MPI_INT, next, 0,      // send part
    bigdata, 1000000, MPI_INT, prev, 0,      // receive part
    MPI_COMM_WORLD, MPI_STATUS_IGNORE);

```

Method 4: Use MPI_Ssend or MPI_Bsend carefully

For debugging or controlled synchronization, you can use:

- `MPI_Ssend` (**synchronous send**): waits until the receiver posts the matching receive.
- `MPI_Bsend` (**buffered send**): you provide an explicit buffer for MPI to copy data into.

```

MPI_Buffer_attach(buffer, buffer_size);
MPI_Bsend(bigdata, 1000000, MPI_INT, next, 0, MPI_COMM_WORLD);
MPI_Buffer_detach(&buffer, &buffer_size);

```

```

int MPI_Send(
    void *buf,           // Address of the data to send
    int count,           // Number of elements to send
    MPI_Datatype datatype, // Type of each element (e.g., MPI_INT)
    int dest,            // Rank of the destination process
    int tag,             // Message tag (to identify the message)
    MPI_Comm comm        // Communicator (usually MPI_COMM_WORLD)
);

```

```

int MPI_Recv(
    void *buf,           // Address where received data will be stored
    int count,           // Maximum number of elements to receive
    MPI_Datatype datatype, // Type of each element
    int source,          // Rank of the sender (can be MPI_ANY_SOURCE)
    int tag,             // Tag to match the message (can be MPI_ANY_TAG)
    MPI_Comm comm,       // Communicator
    MPI_Status *status // Information about the received message (can use
MPI_STATUS_IGNORE)
);

```

Nearest Neighbour Exchange of Data in Ring Topology

```

#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    int rank, size;
    int send_data, recv_data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2) {
        if (rank == 0) printf("Run with at least 2 processes.\n");
        MPI_Finalize();
        return 0;
    }

    send_data = rank;
    int next = (rank + 1) % size;
    int prev = (rank + size - 1) % size;

```

```

    if (rank % 2 == 0) {
        MPI_Send(&send_data, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
        MPI_Recv(&recv_data, 1, MPI_INT, prev, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
    } else {
        MPI_Recv(&recv_data, 1, MPI_INT, prev, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        MPI_Send(&send_data, 1, MPI_INT, next, 0, MPI_COMM_WORLD);
    }

    printf("Process %d sent %d to process %d and received %d from process
%d\n",
        rank, send_data, next, recv_data, prev);

    MPI_Finalize();
    return 0;
}

```

Find Sum of Array Elements by Two Processes

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 2) {
        if (rank == 0) printf("This program requires exactly 2
processes.\n");
        MPI_Finalize();
        return 0;
    }

    int n = 10;
    int A[10] = {1,2,3,4,5,6,7,8,9,10};
    int local_sum = 0;

    if (rank == 0) {
        for (int i = 0; i < n/2; i++)
            local_sum += A[i];
        MPI_Send(&local_sum, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rank == 1) {

```

```

        for (int i = n/2; i < n; i++)
            local_sum += A[i];
        int sum0;
        MPI_Recv(&sum0, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        int total_sum = local_sum + sum0;
        printf("Sum of array elements = %d\n", total_sum);
    }

    MPI_Finalize();
    return 0;
}

```

```

int MPI_Scatter(
    const void *sendbuf, // data to send (only used by root)
    int sendcount,       // number of elements sent to each process
    MPI_Datatype sendtype, // data type of elements sent
    void *recvbuf,       // buffer to receive data
    int recvcnt,         // number of elements received
    MPI_Datatype recvtype, // data type of elements received
    int root,            // rank of the sending process
    MPI_Comm comm        // communicator (usually MPI_COMM_WORLD)
);

```

```

int MPI_Gather(
    const void *sendbuf, // data to send (each process provides its local
data)
    int sendcount,       // number of elements to send
    MPI_Datatype sendtype, // data type of elements sent
    void *recvbuf,       // buffer to receive all data (only used by root)
    int recvcnt,         // number of elements expected from each process
    MPI_Datatype recvtype, // data type of elements received
    int root,            // rank of receiving process
    MPI_Comm comm        // communicator
);

```

****Matrix - Vector Addition**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

```

```

static int header_printed = 0;

int main(int argc, char *argv[]) {
    int rank, size, N;
    int *A = NULL, *x = NULL, *local_rows, *local_result, *final_result;
    int rows_per_proc;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc < 2) {
        if (rank == 0) printf("Usage: %s <matrix_size>\n", argv[0]);
        MPI_Finalize();
        return 0;
    }

    N = atoi(argv[1]);
    if (N % size != 0) {
        if (rank == 0)
            printf("Error: N (%d) must be divisible by number of processes (%d)\n", N, size);
        MPI_Finalize();
        return 0;
    }

    rows_per_proc = N / size;

    x = (int *)malloc(N * sizeof(int));
    local_rows = (int *)malloc(rows_per_proc * N * sizeof(int));
    local_result = (int *)malloc(rows_per_proc * sizeof(int));

    if (rank == 0) {
        A = (int *)malloc(N * N * sizeof(int));
        final_result = (int *)malloc(N * sizeof(int));

        for (int i = 0; i < N; i++) {
            x[i] = i + 1;
            for (int j = 0; j < N; j++) {
                A[i * N + j] = (i + j + 1) % 10;
            }
        }
    }

    MPI_Bcast(x, N, MPI_INT, 0, MPI_COMM_WORLD);

    double start_time = MPI_Wtime();

    MPI_Scatter(A, rows_per_proc * N, MPI_INT,
                local_rows, rows_per_proc * N, MPI_INT,

```

```

        0, MPI_COMM_WORLD);

for (int i = 0; i < rows_per_proc; i++) {
    local_result[i] = 0;
    for (int j = 0; j < N; j++) {
        local_result[i] += local_rows[i * N + j] * x[j];
    }
}

MPI_Gather(local_result, rows_per_proc, MPI_INT,
          final_result, rows_per_proc, MPI_INT,
          0, MPI_COMM_WORLD);

double end_time = MPI_Wtime();
double exec_time = end_time - start_time;

if (rank == 0) {
    FILE *fp = fopen("PS_1.csv", "a");
    if (fp == NULL) {
        printf("Error opening file!\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    fprintf(fp, "%d,%d,%f\n", N, size, exec_time);
    fclose(fp);
}

free(x);
free(local_rows);
free(local_result);
if (rank == 0) {
    free(A);
    free(final_result);
}

MPI_Finalize();
return 0;
}

```

****Matrix - Matrix Multiplication**

```

#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int rank, size, N;

```



```

int *A = NULL, *B = NULL, *local_A, *local_C, *C = NULL;
int rows_per_proc;

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

if (argc < 2) {
    if (rank == 0) printf("Usage: %s <matrix_size>\n", argv[0]);
    MPI_Finalize();
    return 0;
}

N = atoi(argv[1]);
if (N % size != 0) {
    if (rank == 0)
        printf("Error: N (%d) must be divisible by number of processes (%d)\n", N, size);
    MPI_Finalize();
    return 0;
}

rows_per_proc = N / size;

local_A = (int *)malloc(rows_per_proc * N * sizeof(int));
local_C = (int *)malloc(rows_per_proc * N * sizeof(int));
B = (int *)malloc(N * N * sizeof(int));

if (rank == 0) {
    A = (int *)malloc(N * N * sizeof(int));
    C = (int *)malloc(N * N * sizeof(int));

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            A[i * N + j] = (i + j + 1) % 10;
            B[i * N + j] = (i + j + 2) % 10;
        }
    }
}

MPI_Bcast(B, N * N, MPI_INT, 0, MPI_COMM_WORLD);

MPI_Scatter(A, rows_per_proc * N, MPI_INT,
            local_A, rows_per_proc * N, MPI_INT,
            0, MPI_COMM_WORLD);

double start_time = MPI_Wtime();

for (int i = 0; i < rows_per_proc; i++) {
    for (int j = 0; j < N; j++) {

```

```

        local_C[i * N + j] = 0;
        for (int k = 0; k < N; k++) {
            local_C[i * N + j] += local_A[i * N + k] * B[k * N + j];
        }
    }
}

MPI_Gather(local_C, rows_per_proc * N, MPI_INT,
          C, rows_per_proc * N, MPI_INT,
          0, MPI_COMM_WORLD);

double end_time = MPI_Wtime();
double exec_time = end_time - start_time;

if (rank == 0) {
    FILE *fp = fopen("PS_2.csv", "a");
    if (!fp) {
        printf("Error opening file!\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    fprintf(fp, "%d,%d,%f\n", N, size, exec_time);
    fclose(fp);
}

free(local_A);
free(local_C);
free(B);
if (rank == 0) {
    free(A);
    free(C);
}

MPI_Finalize();
return 0;
}

```

Convolution Program

```

#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <string.h>
#include <mpi.h>

#define DEFAULT_ITERATIONS 1

```

```

int conv_column(int *, int, int, int, int *, int);
int conv(int *, int, int, int, int *, int);
int * check(int *, int, int, int *, int);

/* Convolve a column-centered kernel on position i inside sub_grid.
   sub_grid is a linearized 2D block with width DIM (columns).
   kernel is the kernel (kernel_dim x kernel_dim). */
int conv_column(int * sub_grid, int i, int nrows, int DIM, int * kernel, int
kernel_dim) {
    int counter = 0;
    int num_pads = (kernel_dim - 1) / 2;
    int center_index = (((kernel_dim - 1) * (kernel_dim + 1)) / 2);

    for (int j = 1; j <= num_pads; j++) {
        counter += sub_grid[i + j * DIM] * kernel[center_index + j *
kernel_dim];
        counter += sub_grid[i - j * DIM] * kernel[center_index - j *
kernel_dim];
    }
    counter += sub_grid[i] * kernel[center_index];
    return counter;
}

/* Convolve full kernel around element i (handles left/right columns too).
*/
int conv(int * sub_grid, int i, int nrows, int DIM, int * kernel, int
kernel_dim) {
    int counter = 0;
    int num_pads = (kernel_dim - 1) / 2;

    // center column
    counter += conv_column(sub_grid, i, nrows, DIM, kernel, kernel_dim);

    // left and right columns inside the same row (ensure we don't cross row
boundary)
    for (int j = 1; j <= num_pads; j++) {
        int row_start = (i / DIM) * DIM;
        int row_end = row_start + DIM - 1;

        if ((i + j) <= row_end) {
            counter += conv_column(sub_grid, i + j, nrows, DIM, kernel,
kernel_dim);
        }
        if ((i - j) >= row_start) {
            counter += conv_column(sub_grid, i - j, nrows, DIM, kernel,
kernel_dim);
        }
    }
    return counter;
}

```

```

/* Compute convolution for all valid positions in sub_grid and return new
block */
int * check(int * sub_grid, int nrows, int DIM, int * kernel, int
kernel_dim) {
    int num_pads = (kernel_dim - 1) / 2;
    int * new_grid = calloc(DIM * nrows, sizeof(int));
    if (!new_grid) {
        fprintf(stderr, "calloc failed in check()\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
    // iterate over the interior part of sub_grid (skip padded rows at top and
bottom)
    for (int i = (num_pads * DIM); i < (DIM * (num_pads + nrows)); i++) {
        int val = conv(sub_grid, i, nrows, DIM, kernel, kernel_dim);
        new_grid[i - (num_pads * DIM)] = val;
    }
    return new_grid;
}

int main ( int argc, char** argv ) {
    int num_procs, ID;
    int num_iterations = DEFAULT_ITERATIONS;
    int DIM, GRID_WIDTH, KERNEL_DIM, KERNEL_SIZE;
    MPI_Status status;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procs);
    MPI_Comm_rank(MPI_COMM_WORLD, &ID);

    if (argc < 3) {
        if (ID == 0) fprintf(stderr, "Usage: %s <matrix_dim> <kernel_dim>
[iterations]\n", argv[0]);
        MPI_Finalize();
        return 0;
    }

    DIM = atoi(argv[1]);
    KERNEL_DIM = atoi(argv[2]);
    if (argc == 4) num_iterations = atoi(argv[3]);

    if (DIM <= 0 || KERNEL_DIM <= 0) {
        if (ID == 0) fprintf(stderr, "DIM and KERNEL_DIM must be positive
integers\n");
        MPI_Finalize();
        return 0;
    }

    GRID_WIDTH = DIM * DIM;
    KERNEL_SIZE = KERNEL_DIM * KERNEL_DIM;

```

```

if (DIM % num_procs != 0) {
    if (ID == 0) fprintf(stderr, "Error: DIM (%d) must be divisible by
number of processes (%d)\n", DIM, num_procs);
    MPI_Finalize();
    return 0;
}

// Allocate main grid on all processes (root will initialize)
int * main_grid = malloc(GRID_WIDTH * sizeof(int));
if (!main_grid) {
    fprintf(stderr, "malloc failed for main_grid\n");
    MPI_Abort(MPI_COMM_WORLD, 1);
}

// initialize grid on root
if (ID == 0) {
    for (int i = 0; i < GRID_WIDTH; i++) main_grid[i] = 1;
} else {
    // others can start with zeros or unknown; root will overwrite when
sending result back
    memset(main_grid, 0, GRID_WIDTH * sizeof(int));
}

// create kernel (all ones for this example)
int * kernel = malloc(KERNEL_SIZE * sizeof(int));
if (!kernel) { fprintf(stderr, "malloc failed for kernel\n");
MPI_Abort(MPI_COMM_WORLD, 1); }
for (int i = 0; i < KERNEL_SIZE; i++) kernel[i] = 1;

int num_pads = (KERNEL_DIM - 1) / 2;

// compute local partitioning
int rows_per_proc = DIM / num_procs;
int start = rows_per_proc * ID;
int end = start + rows_per_proc - 1;
int nrows = rows_per_proc;

int next = (ID + 1) % num_procs;
int prev = (ID == 0) ? (num_procs - 1) : (ID - 1);

// temporary buffers for halo rows
int halo_size = DIM * num_pads;
int *upper = malloc(sizeof(int) * halo_size);
int *lower = malloc(sizeof(int) * halo_size);
int *pad_row_upper = malloc(sizeof(int) * halo_size);
int *pad_row_lower = malloc(sizeof(int) * halo_size);

if (!upper || !lower || !pad_row_upper || !pad_row_lower) {
    fprintf(stderr, "malloc failed for halo buffers\n");
}

```

```

    MPI_Abort(MPI_COMM_WORLD, 1);
}

double start_time = MPI_Wtime();

for (int iters = 0; iters < num_iterations; ++iters) {

    // copy local boundary rows from main_grid to send buffers
    // lower: the last num_pads rows of this partition
    memcpy(lower, &main_grid[DIM * (end - num_pads + 1)], sizeof(int) *
halo_size);
    // upper: the first num_pads rows of this partition
    memcpy(upper, &main_grid[DIM * start], sizeof(int) * halo_size);

    // Halo exchange with neighbor processes.
    // We use tag pairs (1 and 0) and odd-even ordering to avoid deadlock.
    if (num_procs > 1) {
        if (ID % 2 == 1) {
            MPI_Recv(pad_row_lower, halo_size, MPI_INT, next, 1, MPI_COMM_WORLD,
&status);
            MPI_Recv(pad_row_upper, halo_size, MPI_INT, prev, 1, MPI_COMM_WORLD,
&status);

            MPI_Send(upper, halo_size, MPI_INT, prev, 0, MPI_COMM_WORLD);
            MPI_Send(lower, halo_size, MPI_INT, next, 0, MPI_COMM_WORLD);
        } else {
            MPI_Send(upper, halo_size, MPI_INT, prev, 1, MPI_COMM_WORLD);
            MPI_Send(lower, halo_size, MPI_INT, next, 1, MPI_COMM_WORLD);

            MPI_Recv(pad_row_lower, halo_size, MPI_INT, next, 0, MPI_COMM_WORLD,
&status);
            MPI_Recv(pad_row_upper, halo_size, MPI_INT, prev, 0, MPI_COMM_WORLD,
&status);
        }
    } else {
        // single process: pad rows are the same as boundaries (or zeros for
edges)
        memcpy(pad_row_lower, lower, sizeof(int) * halo_size);
        memcpy(pad_row_upper, upper, sizeof(int) * halo_size);
    }

    // Build local sub_grid = [pad_upper | local_rows | pad_lower]
    int sub_size = DIM * (nrows + 2 * num_pads);
    int * sub_grid = malloc(sizeof(int) * sub_size);
    if (!sub_grid) { fprintf(stderr, "malloc failed for sub_grid\n");
MPI_Abort(MPI_COMM_WORLD, 1); }

    // If first process, pad upper with zeros (no neighbor above)
    if (ID == 0) memset(pad_row_upper, 0, sizeof(int) * halo_size);
    // If last process, pad lower with zeros (no neighbor below)

```

```

    if (ID == num_procs - 1) memset(pad_row_lower, 0, sizeof(int) *
halo_size);

    memcpy(sub_grid, pad_row_upper, sizeof(int) * halo_size);
    memcpy(&sub_grid[halo_size], &main_grid[DIM * start], sizeof(int) * DIM
* nrows);
    memcpy(&sub_grid[halo_size + DIM * nrows], pad_row_lower, sizeof(int) *
halo_size);

    // compute convolution on local subgrid
    int * changed_subgrid = check(sub_grid, nrows, DIM, kernel, KERNEL_DIM);
    free(sub_grid);

    // Gather results at root: send changed_subgrid to root; root collects
and then broadcasts full grid
    if (ID != 0) {
        MPI_Send(changed_subgrid, nrows * DIM, MPI_INT, 0, 11,
MPI_COMM_WORLD);
        // receive updated main grid from root for next iteration
        MPI_Recv(main_grid, GRID_WIDTH, MPI_INT, 0, 10, MPI_COMM_WORLD,
&status);
    } else {
        // root copies its own portion into main_grid
        memcpy(main_grid, changed_subgrid, sizeof(int) * nrows * DIM);

        // receive from other ranks and place in the correct offset
        for (int k = 1; k < num_procs; k++) {
            int offset = DIM * rows_per_proc * k;
            MPI_Recv(&main_grid[offset], nrows * DIM, MPI_INT, k, 11,
MPI_COMM_WORLD, &status);
        }

        // send full updated main_grid to all other processes for next
iteration
        for (int k = 1; k < num_procs; k++) {
            MPI_Send(main_grid, GRID_WIDTH, MPI_INT, k, 10, MPI_COMM_WORLD);
        }
    }

    free(changed_subgrid);
}

double end_time = MPI_Wtime();
if (ID == 0) {
    printf("Execution Time: %f seconds (DIM=%d, kernel=%d, procs=%d)\n",
end_time - start_time, DIM, KERNEL_DIM, num_procs);
}

// free allocated memory
free(main_grid);

```

```
free(kernel);
free(upper);
free(lower);
free(pad_row_upper);
free(pad_row_lower);

MPI_Finalize();
return 0;
}
```

Dot Product

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[]) {
    int rank, size, N;
    double *vector_a = NULL, *vector_b = NULL;
    double *local_a, *local_b;
    double local_dot_product, global_dot_product;
    int elements_per_proc;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc < 2) {
        if (rank == 0) printf("Usage: %s <vector_size>\n", argv[0]);
        MPI_Finalize();
        return 0;
    }

    N = atoi(argv[1]);

    if (N % size != 0) {
        if (rank == 0)
            printf("Error: N (%d) must be divisible by number of processes (%d)\n", N, size);
        MPI_Finalize();
        return 0;
    }

    elements_per_proc = N / size;
```



```

local_a = (double *)malloc(elements_per_proc * sizeof(double));
local_b = (double *)malloc(elements_per_proc * sizeof(double));

if (rank == 0) {
    vector_a = (double *)malloc(N * sizeof(double));
    vector_b = (double *)malloc(N * sizeof(double));

    srand(time(NULL));
    for (int i = 0; i < N; i++) {
        vector_a[i] = (double)(rand() % 100) / 10.0;
        vector_b[i] = (double)(rand() % 100) / 10.0;
    }

    FILE *fp = fopen("PS_2.csv", "r");
    if (fp == NULL) {
        fp = fopen("PS_2.csv", "w");
        if (fp != NULL) {
            fprintf(fp, "VectorSize,NumProcesses,TimeTaken\n");
            fclose(fp);
        }
    } else {
        fclose(fp);
    }
}

double start_time = MPI_Wtime();

MPI_Scatter(vector_a, elements_per_proc, MPI_DOUBLE,
            local_a, elements_per_proc, MPI_DOUBLE,
            0, MPI_COMM_WORLD);

MPI_Scatter(vector_b, elements_per_proc, MPI_DOUBLE,
            local_b, elements_per_proc, MPI_DOUBLE,
            0, MPI_COMM_WORLD);

local_dot_product = 0.0;
for (int i = 0; i < elements_per_proc; i++) {
    local_dot_product += local_a[i] * local_b[i];
}

MPI_Reduce(&local_dot_product, &global_dot_product, 1, MPI_DOUBLE,
            MPI_SUM, 0, MPI_COMM_WORLD);

double end_time = MPI_Wtime();
double exec_time = end_time - start_time;

if (rank == 0) {
    FILE *fp = fopen("PS_2.csv", "a");
    if (fp == NULL) {
        MPI_Abort(MPI_COMM_WORLD, 1);
    }
}

```

```

    }
    fprintf(fp, "%d,%d,%f\n", N, size, exec_time);
    fclose(fp);
}

free(local_a);
free(local_b);

if (rank == 0) {
    free(vector_a);
    free(vector_b);
}

MPI_Finalize();
return 0;
}

```

N - Queens

```

#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define MAX_THREADS 16
#define MAX_SIZES 3

int is_safe(int *board, int row, int col) {
    for (int i = 0; i < row; i++) {
        if (board[i] == col || abs(board[i] - col) == abs(i - row)) {
            return 0;
        }
    }
    return 1;
}

void solve_nqueens(int *board, int row, int N, long *solutions) {
    if (row == N) {
        #pragma omp atomic
        (*solutions)++;
        return;
    }

    for (int col = 0; col < N; col++) {
        if (is_safe(board, row, col)) {
            board[row] = col;
            solve_nqueens(board, row + 1, N, solutions);
        }
    }
}

```

```

    }
}

long parallel_nqueens(int N, int threads) {
    long solutions = 0;

    #pragma omp parallel num_threads(threads)
    {
        int *board = (int *)malloc(N * sizeof(int));
        int tid = omp_get_thread_num();

        #pragma omp for schedule(dynamic)
        for (int col = 0; col < N; col++) {
            board[0] = col;
            solve_nqueens(board, 1, N, &solutions);
        }
        free(board);
    }
    return solutions;
}

int main() {
    int sizes[MAX_SIZES] = {8, 10, 12};
    FILE *fp = fopen("PS_1_A.csv", "a");

    if (fp == NULL) {
        printf("Error opening PS_1_A.csv!\n");
        return 1;
    }

    fseek(fp, 0, SEEK_END);
    if (ftell(fp) == 0) {
        fprintf(fp, "BoardSize,Threads,ExecutionTime,Solutions\n");
    }

    printf("%-10s %-7s %-12s %-12s\n", "BoardSize", "Threads", "Time(s)",
        "Solutions");
    printf("-----\n");

    for (int s = 0; s < MAX_SIZES; s++) {
        int N = sizes[s];

        for (int t = 1; t <= MAX_THREADS; t *= 2) {
            double start_time = omp_get_wtime();
            long solutions = parallel_nqueens(N, t);
            double end_time = omp_get_wtime();

            double exec_time = end_time - start_time;

```

```
        printf("%-10d %-7d %-12.6f %-12ld\n", N, t, exec_time,
solutions);
        fprintf(fp, "%d,%d,%f,%ld\n", N, t, exec_time, solutions);
    }
}

fclose(fp);
return 0;
}
```
